



This migrate command will look out for all the `INSTALLED_APPS` settings. It will then create any important database tables depending on the database settings as per your `mysite/settings.py` file.

For each migration, there will be a message displayed. If it is important for you, then you can run this specific command-lines client for that database: Type `\dt (PostgreSQL) -> SHOW TABLES;` (MariaDB, MySQL), `.schema (SQLite)`, or `SELECT TABLE_NAME FROM USER_TABLES;` (Oracle) in order to display the tables in the Django created.

Creating models

Now the next step is to define models. It will cover a database layout along with additional metadata. By theory, a model is the only place of information related to your data. This model contains all the required fields and also stores the behaviour of your data that is getting stored.

So, we are building a Poll app using Django. In this poll app, we have to create two models: First – ‘Question’ and second is the ‘Choice’. The Question model will contain a question with publication details. Whereas, the Choice model will have two fields defined: a vote tally & choice text. Here, each Choice will be associated with some Question.

All these concepts are represented by a set of Python classes. You have to Edit ‘`polls/models.py`’ files. The image on the next page is the displayed version of it.

Here, each model is represented by a class that then subclasses `django.db.models.Model`. There are a number of class variables in each set of models. The class variables represent database fields in models. Each field is further represented by the field class instance. For example, `CharField` is for character-related fields & `DateTimeField` is for Date and time. Through this, Django has a clear understanding of data types that each of the fields holds in it. In the machine-friendly formats, field instance names are the fields. Example – `question_text` or `pub_date`. All